

Contents

Introduction to MySQL 3

 Overview: What is MySQL?..... 3

 MySQL vs. MariaDB 3

 MySQL Architecture & Client-Server Model 3

 Storage Engines (InnoDB vs. MyISAM) 3

 Installation Guides..... 3

 A. Installation for Windows..... 3

 B. Installation for Ubuntu 4

SQL Basics 4

 Creating Databases & Tables (DDL) 4

 CRUD Operations (DML)..... 5

 User & Access Management 6

 Hands-On Lab 7

 Connecting Data (JOINS) 8

 Performance Optimization: Indexing..... 9

Database Server Configuration..... 10

 Important Config Files 10

 Key Configuration Directives 10

 Securing the Installation (mysql_secure_installation) 11

Backup & Recovery..... 11

 Logical Backups with mysqldump 11

 Restoring Databases 12

 Automating Backups with Cron 12

Performance & Monitoring 13

 Understanding Logs 13

 Using EXPLAIN for Query Analysis 13

 ACID Principles & Transactions 14

Security & High Availability (Overview) 14

 Principle of Least Privilege 14

Enabling SSL Connections.....	15
Basics of Replication (Master-Slave).....	15
Setting Up Replication	15
Essential Replication Commands	17
Replication Test: The "Disaster" Test.....	18

Introduction to MySQL

Overview: What is MySQL?

MySQL is the world's most popular open-source Relational Database Management System (RDBMS). It organizes data into tables (rows and columns) and uses Structured Query Language (SQL) to manage that data. It is widely used for web applications (like Facebook, WordPress) due to its speed, reliability, and ease of use.

MySQL vs. MariaDB

MariaDB is a fork of MySQL created by the original developers of MySQL after Oracle acquired it. They are highly compatible but have diverged slightly.

Feature	MySQL	MariaDB
Owner	Oracle Corporation	MariaDB Foundation (Open Source community)
Licensing	GPL + Proprietary Enterprise Edition	Truly Open Source (GPL)
Performance	High performance, excellent thread pooling in Enterprise	Often faster replication and query execution in open versions
Updates	Slower release cycle	Faster, more frequent community updates

MySQL Architecture & Client-Server Model

MySQL operates on a Client-Server model.

1. The Server (mysqld): The background process that manages the database files, processes queries, and handles security.
2. The Client: Programs that connect to the server to issue requests (e.g., MySQL Workbench, command line mysql, PHP scripts).

Storage Engines (InnoDB vs. MyISAM)

Storage engines handle how data is stored, retrieved, and locked.

- InnoDB (Default): Supports Transactions (ACID compliant), Row-level locking (better for multi-user concurrency), and Foreign Keys. Use this for modern applications.
- MyISAM (Legacy): Faster for read-heavy operations but supports only Table-level locking (writing to a table locks the whole thing) and does not support transactions.

Installation Guides

A. Installation for Windows

1. Download the MySQL Installer from dev.mysql.com.
2. Run the installer and choose "Custom" or "Developer Default".
3. Select MySQL Server and MySQL Workbench.

4. Click Next to install packages.
5. Configuration:
 - Config Type: Development Computer.
 - Port: 3306 (Default).
 - Auth Method: Use Strong Password Encryption.
6. Set your Root Password (Write this down!).
7. Execute the configuration and Click Finish.

B. Installation for Ubuntu

1. Update package index: `sudo apt update`
2. Install the server: `sudo apt install mysql-server`
3. Start the service: `sudo systemctl start mysql`
4. Run the security script (crucial step): `sudo mysql_secure_installation`
 - *Follow prompts to set root password, remove anonymous users, and disable remote root login.*
5. Log in: `sudo mysql -u root -p`

SQL Basics

Creating Databases & Tables (DDL)

Concepts:

- Data Types: INT, VARCHAR(n) (string), DECIMAL(10,2) (money), DATE, BOOLEAN.
- Constraints: Rules for data.
 - NOT NULL: Cannot be empty.
 - UNIQUE: No duplicates allowed.
 - AUTO_INCREMENT: Automatically adds 1 to the previous ID.
 - PRIMARY KEY: Unique identifier for the row.
 - FOREIGN KEY: Links to another table's Primary Key.

Examples

```
// Create a new database
CREATE DATABASE inventory_db;
```

```
// Select the database to use
USE inventory_db;
```

```
// Create a 'Categories' table (Parent table)
CREATE TABLE categories (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL UNIQUE
);
```

```
// Create a 'Products' table (Child table with Foreign Key)
CREATE TABLE products (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(150) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  stock_qty INT DEFAULT 0,
  category_id INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

```
// Create a 'Customers' table
CREATE TABLE customers (
  cust_id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  full_name VARCHAR(100)
);
```

```
// Add a column to an existing table
ALTER TABLE customers ADD COLUMN phone VARCHAR(20);
```

```
// Modify a column definition
ALTER TABLE products MODIFY COLUMN price DECIMAL(12, 2);
```

```
// Rename a table
RENAME TABLE customers TO clients;
```

```
// Drop a specific column
ALTER TABLE clients DROP COLUMN phone;
```

```
// Delete a table entirely
DROP TABLE IF EXISTS old_backup_table;
```

CRUD Operations (DML)

Create (INSERT), Read (SELECT), Update, Delete.

```
// INSERT: Single record
INSERT INTO categories (category_name) VALUES ('Electronics');
```

```
// INSERT: Multiple records at once
INSERT INTO categories (category_name) VALUES ('Furniture'), ('Clothing'), ('Toys');
```

```
// INSERT: Into products
INSERT INTO products (product_name, price, stock_qty, category_id)
VALUES ('Gaming Laptop', 1500.00, 10, 1), ('Office Chair', 150.50, 25, 2);
```

```
// SELECT: View all columns
SELECT * FROM products;
```

```
// SELECT: Specific columns with Filtering
SELECT product_name, price FROM products WHERE price > 500;
```

```
// SELECT: Sorting and Limiting
SELECT * FROM products ORDER BY price DESC LIMIT 5;
```

```
// SELECT: Using simple calculation
SELECT product_name, price * 0.9 AS discounted_price FROM products;
```

```
// UPDATE: Change a specific record
UPDATE products SET stock_qty = 15 WHERE product_id = 1;
```

```
// UPDATE: Bulk update (Safe mode must be off or use Key)
UPDATE products SET price = price * 1.10 WHERE category_id = 2; -- 10% price hike for furniture
```

```
// DELETE: Remove specific record
DELETE FROM categories WHERE category_name = 'Toys';
```

User & Access Management

Securing your database involves creating specific users rather than using root for everything.

Concepts

- Host: Defines where the user can connect from. localhost (this machine only) or % (any IP address).
- Privileges: ALL PRIVILEGES, SELECT, INSERT, UPDATE, DELETE, CREATE, etc.
- Roles (MySQL 8.0+): A named collection of privileges.

```
// Create a user that can only connect from the local machine
CREATE USER 'local_dev'@'localhost' IDENTIFIED BY 'password123';
```

```
// Create a user that can connect from ANY IP address (Remote access)
CREATE USER 'remote_admin'@'%' IDENTIFIED BY 'securePass!';
```

```
// Grant ALL privileges on ALL databases (Superuser behavior - Use carefully)
GRANT ALL PRIVILEGES ON *.* TO 'remote_admin'@'%';
```

```
// Grant specific privileges on a specific database
GRANT SELECT, INSERT, UPDATE ON inventory_db.* TO 'local_dev'@'localhost';
```

```
// Grant permission to only SELECT from one specific table
GRANT SELECT ON inventory_db.products TO 'intern_user'@'localhost';
```

```
// Show the privileges defined for a user
SHOW GRANTS FOR 'local_dev'@'localhost';
```

```
// Revoke specific permissions (Take away Insert rights)
REVOKE INSERT ON inventory_db.* FROM 'local_dev'@'localhost';
```

```
// Delete a user
DROP USER 'intern_user'@'localhost';
```

```
// FLUSH PRIVILEGES (Sometimes required to apply changes immediately)
FLUSH PRIVILEGES;
```

Hands-On Lab

We will build a mini-Library system. We need a table for Books and a Librarian user who can manage them.

Step 1: Create Schema

```
CREATE DATABASE library_sys;
USE library_sys;

CREATE TABLE books (
  book_id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  author VARCHAR(100),
  published_year YEAR,
  is_available BOOLEAN DEFAULT TRUE
);
```

Step 2: Insert Data

```
INSERT INTO books (title, author, published_year) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925),
('1984', 'George Orwell', 1949),
('To Kill a Mockingbird', 'Harper Lee', 1960),
('The Hobbit', 'J.R.R. Tolkien', 1937),
('Harry Potter', 'J.K. Rowling', 1997);
```

Step 3: Query Data

```
// Find books published before 1950
SELECT title, author FROM books WHERE published_year < 1950;
```

```
// Count how many books we have
SELECT COUNT(*) AS total_books FROM books;
```

```
// Mark '1984' as borrowed (not available)
UPDATE books SET is_available = FALSE WHERE title = '1984';
```

Step 4: Manage Users

We want a user `librarian_tom` who can view and update books, but cannot delete them or drop tables.

```
// Create the user
CREATE USER 'librarian_tom'@'localhost' IDENTIFIED BY 'tom_pass';

// Grant specific permissions
GRANT SELECT, INSERT, UPDATE ON library_sys.books TO 'librarian_tom'@'localhost';

// Test: Verify Tom cannot delete
// (You would log in as Tom to test this, but the syntax to revoke just in case he had it is:)
REVOKE DELETE ON library_sys.books FROM 'librarian_tom'@'localhost';
```

Connecting Data (JOINS)

JOINS are used to combine rows from these two or more tables based on that related column.

Overview: Types of Joins

- INNER JOIN: Returns records that have matching values in *both* tables.
- LEFT JOIN: Returns *all* records from the left table, and the matched records from the right table (returns NULL if no match).
- RIGHT JOIN: The opposite of Left Join.
- CROSS JOIN: Returns all records from both tables (Cartesian product—rarely used efficiently).

Assuming we are using the `inventory_db` created in the previous step.

1. INNER JOIN

- Show product name and its category name.
- Only shows products that HAVE a category assigned.

```
SELECT products.product_name, categories.category_name
FROM products
INNER JOIN categories ON products.category_id = categories.category_id;
```


2. LEFT JOIN

- Show ALL products, even if they don't have a category (category will show as NULL).
- Good for finding "orphaned" data.

```
SELECT p.product_name, c.category_name
FROM products p -- (Using 'p' as an alias for brevity)
LEFT JOIN categories c ON p.category_id = c.category_id;
```

3. RIGHT JOIN

- Show ALL categories, even if no products belong to them.
- Good for finding empty categories.

```
SELECT c.category_name, p.product_name
FROM products p
RIGHT JOIN categories c ON p.category_id = c.category_id;
```

4. JOIN with WHERE clause

- Show only Electronics products that cost more than 1000.

```
SELECT p.product_name, p.price, c.category_name
FROM products p
INNER JOIN categories c ON p.category_id = c.category_id
WHERE c.category_name = 'Electronics' AND p.price > 1000;
```

5. JOIN with AGGREGATE functions

- Count how many products are in each category.

```
SELECT c.category_name, COUNT(p.product_id) as total_products
FROM categories c
LEFT JOIN products p ON c.category_id = p.category_id
GROUP BY c.category_name;
```

Performance Optimization: Indexing

As your tables grow from 10 rows to 1,000,000 rows, queries get slow. Indexes are like the "Index" at the back of a book—they let MySQL find data without reading every single page (Full Table Scan).

Overview

- Primary Key: Automatically indexed.
- Unique Index: Enforces uniqueness and speeds up search.
- Standard Index: Just speeds up search (good for columns used in WHERE, ORDER BY, or JOIN).

Examples

```
// Create a simple index on a column frequently searched (e.g., email)
// This makes "SELECT * FROM customers WHERE email = '...'" much faster.
CREATE INDEX idx_email ON customers(email);
```

```
// Create a Multi-Column (Composite) Index
// Useful if you often search by both First AND Last name together.
CREATE INDEX idx_full_name ON customers(last_name, first_name);
```

```
// Create a Unique Index (prevents duplicates)
// This ensures no two products have the exact same SKU code.
CREATE UNIQUE INDEX idx_sku ON products(sku_code);
```

```
// View existing indexes on a table
SHOW INDEX FROM customers;
```

```
// Drop an index (if it's not needed or slowing down INSERTs)
// Note: Indexes speed up READS but slightly slow down WRITES (INSERT/UPDATE).
DROP INDEX idx_email ON customers;
```

Database Server Configuration

Important Config Files

MySQL reads startup options from configuration files.

- Linux/Ubuntu: Usually located at `/etc/mysql/my.cnf` or `/etc/mysql/mysql.conf.d/mysqld.cnf`.
- Windows: Usually named `my.ini` located in the hidden ProgramData folder (e.g., `C:\ProgramData\MySQL\MySQL Server 8.0\`).

Key Configuration Directives

You edit these files to tune performance or change network settings. *Always restart the service after changes (`sudo systemctl restart mysql`).*

1. `bind-address = 0.0.0.0`:
 - By default (usually `127.0.0.1`), MySQL only accepts local connections. Changing this to `0.0.0.0` allows remote connections from *any* IP (requires firewall adjustment).
2. `port = 3306`:
 - The default listening port. Changing this is a "security through obscurity" tactic to reduce automated bot attacks.
3. `max_connections = 151`:
 - Limits how many users can connect simultaneously. Increase this for high-traffic web apps (e.g., set to 500 or 1000).
4. `slow_query_log = 1`:
 - Enables logging of queries that take longer than a specific time. Essential for optimization.
5. `datadir = /var/lib/mysql`:
 - Defines where the actual data files are stored. You might move this if your primary disk fills up.

Securing the Installation (mysql_secure_installation)

This is a shell script included with MySQL Server. It runs a series of interactive prompts to lock down defaults.

Actions performed by the script:

1. Set Root Password: Ensures the superuser has a password.
2. Validate Password Plugin: Optionally enforces strict password complexity rules (length, symbols, etc.).
3. Remove Anonymous Users: Deletes the default "blank" user that lets anyone log in without an account.
4. Disallow Root Login Remotely: Ensures root can only log in from localhost. (Best practice: Create a separate admin user for remote access).
5. Remove Test Database: Deletes the default test database which often has loose permissions.

Backup & Recovery

Logical Backups with mysqldump

```
// Standard Backup (Review)
mysqldump -u root -p my_database > backup.sql
```

```
// Consistent Backup for InnoDB (Crucial for live databases)
// --single-transaction: Creates a backup without locking the tables, keeping the site live.
mysqldump -u root -p --single-transaction --quick my_database > live_backup.sql
```

```
// Backup Stored Procedures and Events
// By default, routines might be skipped. Use this to ensure logic is saved.
mysqldump -u root -p --routines --events my_database > full_logic_backup.sql
```

```
// Separate Data and Structure
// Export only the table creation scripts (no data).
mysqldump -u root -p --no-data my_database > schema_only.sql
```

```
// Compressed Backup (For large databases)
// Pipes the output directly to gzip to save disk space.
mysqldump -u root -p my_database | gzip > backup.sql.gz
```

Restoring Databases

1. Create the target database (if it doesn't exist):

```
mysqladmin -u root -p create new_db_name
```

2. Run the restore command:

```
mysql -u root -p new_db_name < backup.sql
```

3. For compressed backups:

```
gunzip < backup.sql.gz | mysql -u root -p new_db_name
```

Automating Backups with Cron

On Linux, cron is a time-based job scheduler.

1. Open the crontab editor: `crontab -e`
2. Add a line to schedule the backup.

Syntax: m h dom mon dow command

Examples

```
// Run every day at 2:30 AM
```

```
30 02 * * * /usr/bin/mysqldump -u root -p'PASSWORD' my_db > /backups/db_backup.sql
```

```
// Run every Monday at 5:00 AM
```

```
00 05 * * 1 /usr/bin/mysqldump -u root -p'PASSWORD' --all-databases > /backups/weekly_full.sql
```

```
// Timestamped filenames (Best Practice)
```

```
// Creates a file like 'db_backup_2025-12-15.sql' so you don't overwrite the previous day.
```

```
00 03 * * * mysqldump -u root -p'PASS' my_db > /backups/db_backup_$(date +%F).sql
```

```
// Run hourly (at minute 0)
```

```
0 * * * * /path/to/backup_script.sh
```

```
// Run every 5 minutes (for high-intensity transaction logs)
```

```
* /5 * * * * /path/to/incremental_backup.sh
```

Note: Placing passwords directly in crontab is insecure. It is better to use a .my.cnf file in the user's home directory with credentials.

Performance & Monitoring

Understanding Logs

To fix issues, you must know what is happening inside the server.

1. Error Log: Records startup/shutdown events and critical errors (e.g., "Table crashed", "Out of memory").
 - Location: `/var/log/mysql/error.log`
2. General Query Log: Records every client connection and SQL statement.
 - Warning: Massive performance hit. Only enable temporarily for debugging.
3. Slow Query Log: Records queries that take longer than `long_query_time` (default 10s).
 - Use: Identify which SQL statements are slowing down your website.

Using EXPLAIN for Query Analysis

EXPLAIN is a keyword you put *before* a SELECT statement. It tells you how MySQL intends to execute the query (which indexes it will use, how many rows it scans).

```
// Basic Usage
EXPLAIN SELECT * FROM users WHERE email = 'test@example.com';
```

```
// Detecting Full Table Scans
// If the output under 'type' says 'ALL', MySQL is scanning every row.
// This usually means you need an Index.
EXPLAIN SELECT * FROM orders WHERE order_date = '2023-01-01';
```

```
// Checking Index Usage
// Look at the 'key' column. If it is NULL, no index is being used.
// If 'key' shows 'idx_lastname', the index is working.
EXPLAIN SELECT * FROM employees WHERE last_name = 'Smith';
```

```
// Analyzing Joins
EXPLAIN SELECT * FROM users u JOIN orders o ON u.id = o.user_id;

// Look for 'type: ref' or 'type: eq_ref' (Good).
// If you see 'Block Nested Loop', the join might be slow.
```

```
// EXPLAIN ANALYZE (MySQL 8.0+)
// Actually runs the query and gives timing data.
EXPLAIN ANALYZE SELECT * FROM products WHERE price > 500;
```

ACID Principles & Transactions

Transactions ensure data integrity. If you transfer money, you want to deduct from Account A and add to Account B. If the power fails halfway, neither should happen.

- Atomicity: All or nothing.
- Consistency: Data remains valid according to rules.
- Isolation: Transactions don't interfere with each other.
- Durability: Once saved, it stays saved even if power is lost.

Transaction Guide

1. Start the transaction

```
START TRANSACTION;
```

2. Execute First Query (Deduct Money)

```
UPDATE accounts SET balance = balance - 100 WHERE user_id = 1;
```

3. Execute Second Query (Add Money)

```
UPDATE accounts SET balance = balance + 100 WHERE user_id = 2;
```

4. Commit (Save Changes)

```
// If everything worked, run this. Both updates become permanent simultaneously.  
COMMIT;
```

OR

5. Rollback (Undo)

```
// If an error occurred in step 2 or 3, run this.  
// The database reverts to the state before 'START TRANSACTION'.  
ROLLBACK;
```

Security & High Availability (Overview)

Principle of Least Privilege

Never give a user more permission than they need.

- Application Users: Should usually only have SELECT, INSERT, UPDATE, DELETE. They should almost never have DROP or ALTER.
- Reporting Users: Should only have SELECT.
- Dev Users: Limited to specific databases, not the whole server.

Enabling SSL Connections

Data sent between the client and server is plain text by default. SSL encrypts this traffic.

1. Check availability: SHOW VARIABLES LIKE '%ssl%'; (Look for have_ssl = YES).
2. Enforce SSL per user:

```
CREATE USER 'secure_user'@'%' IDENTIFIED BY 'pass' REQUIRE SSL;
```

Now, this user cannot connect unless their client supports and requests encryption.

Basics of Replication (Master-Slave)

Replication copies data from one server (Master) to another (Slave) in near real-time.

Concepts:

1. Master (The Source): Handles all Writes (INSERT, UPDATE, DELETE). It logs these changes to a file called the Binary Log.
2. Slave (The Replica): Connects to the Master, reads the Binary Log, and executes the same SQL commands on itself.
3. Use Cases:
 - Scale Reads: Direct all SELECT queries to Slaves to reduce load on the Master.
 - Backup: Run heavy backups on the Slave so the Master performance isn't affected.
 - Failover: If Master dies, promote the Slave to be the new Master.

Setting Up Replication

Scenario:

- Master Server (Source): IP 192.168.1.10 (Where writes happen)
- Slave Server (Replica): IP 192.168.1.20 (Where data is copied to)

Phase 1: Configure the Master (Source)

1. Edit Configuration File: Open the MySQL config file (usually /etc/mysql/mysql.conf.d/mysqld.cnf) on the Master. You must set a unique ID and enable the binary log (the record of all changes).

```
[mysqld]
bind-address      = 0.0.0.0                # Allow remote connections
server-id         = 1                      # Must be unique
log_bin           = /var/log/mysql/mysql-bin.log
binlog_do_db      = inventory_db           # (Optional) Only replicate this DB
```

2. Restart MySQL:

```
sudo systemctl restart mysql
```

3. Create a Replication User: Log in to MySQL on the Master and create a user specifically for the Slave to use.

```
CREATE USER 'repl_user'@'%' IDENTIFIED BY 'secure_password';
GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'%';
FLUSH PRIVILEGES;
```

4. Lock Tables & Get Status: To ensure the snapshot matches the log position, lock the database momentarily (read-only mode).

```
FLUSH TABLES WITH READ LOCK;
SHOW MASTER STATUS;
```

Keep this terminal open! Write down the File (e.g., mysql-bin.000001) and Position (e.g., 890). You will need these for the Slave.

5. Dump Data (If needed): If the Master already has data, open a *new* terminal window and dump it.

```
mysqldump -u root -p --all-databases --master-data > master_dump.sql
```

6. Unlock Tables: Back in the first MySQL terminal:

```
UNLOCK TABLES;
```

Phase 2: Configure the Slave (Replica)

1. Edit Configuration File: Open the config file on the Slave.

```
[mysqld]
server-id      = 2                                # Must be different from Master
relay-log      = /var/log/mysql/mysql-relay-bin.log
```

2. Restart MySQL:

```
sudo systemctl restart mysql
```

3. Import Data: Copy the master_dump.sql file from the Master to the Slave and import it.

```
mysql -u root -p < master_dump.sql
```

Phase 3: Connect Slave to Master

1. Configure the Link: Log in to MySQL on the Slave and run the following command using the details you wrote down in Phase 1 (Step 4).

```
CHANGE MASTER TO
MASTER_HOST='192.168.1.10',
MASTER_USER='repl_user',
MASTER_PASSWORD='secure_password',
MASTER_LOG_FILE='mysql-bin.000001',      -- From SHOW MASTER STATUS
MASTER_LOG_POS=890;                      -- From SHOW MASTER STATUS
```


2. Start Replication:

```
START SLAVE;
```

3. Verify Status:

```
SHOW SLAVE STATUS\G
```

Check these two fields:

- Slave_IO_Running: Yes (Connected to Master)
- Slave_SQL_Running: Yes (Executing changes)
- *If either is "No", check the Last_IO_Error field.*

Essential Replication Commands

Here are the most common commands you will use to manage and troubleshoot your replication setup.

```
// Check Slave Health (The most important command)
// Displays the detailed status of the replication threads.
SHOW SLAVE STATUS\G
```

```
// Stop Replication (Temporarily) Pauses the replication.
// Useful if you need to patch the server or perform maintenance.
STOP SLAVE;
```

```
// Resume Replication Restarts the replication threads after a stop.
START SLAVE;
```

```
// Skip a Replication Error If the Slave gets stuck on a specific error
// (e.g., trying to delete a row that doesn't exist) and you just want to ignore it to keep going.
STOP SLAVE;
SET GLOBAL SQL_SLAVE_SKIP_COUNTER = 1; -- Skip 1 event
START SLAVE;
```

```
// Show Connected Slaves (On Master) See which slaves are currently connected to this Master.
SHOW SLAVE HOSTS;
```

```
// Show Master Logs List all binary logs on the Master. Useful for disk space management.
SHOW BINARY LOGS;
```

```
// Purge Old Logs Delete binary logs older than a certain date to free up disk space on the Master.
PURGE BINARY LOGS BEFORE '2025-01-01 00:00:00';
```

```
// Reset Slave Warning: This makes the Slave forget its position.  
// Only use this if you are re-initializing replication from scratch.  
STOP SLAVE;  
RESET SLAVE;
```

```
// Check Log Configuration Verify that binary logging is actually on.  
SHOW VARIABLES LIKE 'log_bin';
```

```
// View Current Master Status Check the current file and position on the Master  
// (usually to set up a new Slave).  
SHOW MASTER STATUS;
```

Replication Test: The "Disaster" Test

To confirm your learning, perform this test on your new Replication setup:

1. On Master: Create a new database called `replication_test`.

```
CREATE DATABASE replication_test;  
CREATE TABLE replication_test.dummy (id INT, message VARCHAR(50));  
INSERT INTO replication_test.dummy VALUES (1, 'Hello Slave');
```

2. On Slave: Immediately check if the data exists.

```
SELECT * FROM replication_test.dummy;
```

(You should see "Hello Slave" immediately.)

3. Simulation: "Unplug" the Slave (Stop the Slave), add data to the Master, then "Plug it back in" (Start Slave). Watch it catch up automatically.